

Classification & Model Selection

AI Accelerator Program

Session 14 | 4:30 PM – 6:30

Instructor;
Mahi Aminu Aliyu
AI & SI Associate AHFID



Learning Objectives

1. **Understand classification fundamentals** — Binary classification problems, metrics beyond accuracy
2. **Explore multiple algorithms** — Logistic Regression, Decision Trees, Random Forests, SVM, Gradient Boosting
3. **Diagnose model performance** — Learning curves reveal overfitting vs underfitting
4. **Hyperparameter tuning** — Validation curves guide complexity selection
5. **Interpret predictions** — Feature importance shows what the model learned



Real-World Context: Nigerian Microfinance

The Challenge: Microfinance institutions in Nigeria make thousands of loan decisions annually.

- **Too conservative?** Reject good customers → missed revenue
- **Too liberal?** Approve risky customers → huge losses

The Cost Asymmetry:

- False Negative (approve bad loan) = **₦500,000 default loss**
- False Positive (reject good loan) = **₦5,000 opportunity cost**

→ *Cost of being wrong ≠ Equal!* This shapes everything we'll do.



Confusion Matrix: The Four Outcomes

Confusion Matrix: Loan Default Prediction

Actual Default	No	TN Correct Approval <i>Good loan Approved ✓</i>	FP Wrong Rejection <i>Good loan Rejected ✗</i>
	Yes	FN Wrong Approval <i>Bad loan Approved ✗ (HIGH COST!)</i>	TP Correct Rejection <i>Bad loan Rejected ✓</i>
		No	Yes

Key: FN (False Negative) is the most expensive error in lending!

Key insight for lending: False Negatives (bottom-right) are the most expensive.

Classification Problem Setup

Binary Classification:

- **Input (X):** Customer features
 - Age, monthly income, loan amount, tenure, interest rate, credit score, employment type, bank account, prior defaults
- **Output (y):** Default or not (0 = repaid, 1 = defaulted)
- **Goal:** Build a function $\hat{y} = f(X)$ that predicts who will default

Why not just use regression? Output must be *probability between 0 and 1*, not unbounded values.



Algorithm 1: Logistic Regression

Idea: Fit an S-shaped curve to map features → probability of default

$$P(\text{default} = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}}$$

Visual intuition:

- Linear regression fits a line → can predict values outside [0,1]
- Logistic regression squashes the line through sigmoid function → always between 0 and 1

Pros: Fast, interpretable, probabilistic output

Cons: Assumes linear relationship after sigmoid transform

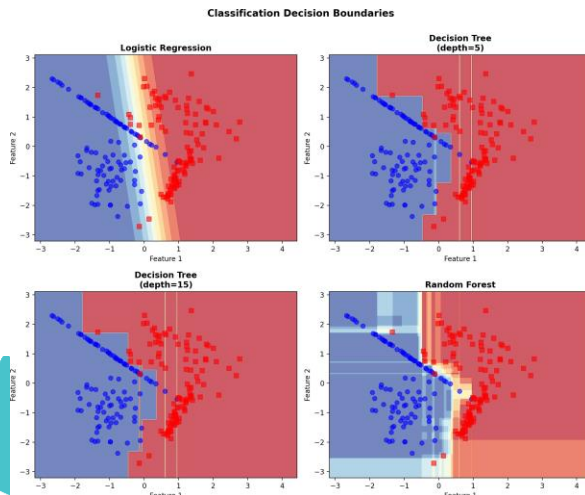


Algorithm 2: Decision Trees

Idea: Ask a series of yes/no questions to partition the data space

```
Is credit_score < 500?  
└ YES → "High risk" (default)  
└ NO → Is income < ₦50k?  
      └ YES → "Medium risk"  
      └ NO → "Low risk"
```

Visual: Each split creates a boundary perpendicular to feature axes.



Algorithm 3: Random Forest

Idea: Build many decision trees on random subsets of data, then average their predictions

Why ensemble?

- Individual trees overfit easily
- Averaging many diverse trees → smoother, more robust predictions
- Reduces variance without increasing bias

Pros: Powerful, handles non-linear patterns, provides feature importance

Cons: Less interpretable than single tree, slower to train



Algorithm 4: Support Vector Machine (SVM)

Idea: Find the best separating line/boundary that maximizes margin between classes

Support Vectors = points closest to boundary (most informative samples)
Maximize margin = keep boundary far from all points
Kernel trick = can handle non-linear boundaries by implicitly transforming data

Pros: Effective in high dimensions, mathematically principled

Cons: Sensitive to feature scaling, harder to interpret



Algorithm 5: Gradient Boosting

Idea: Build trees *sequentially*, where each new tree corrects errors of previous trees

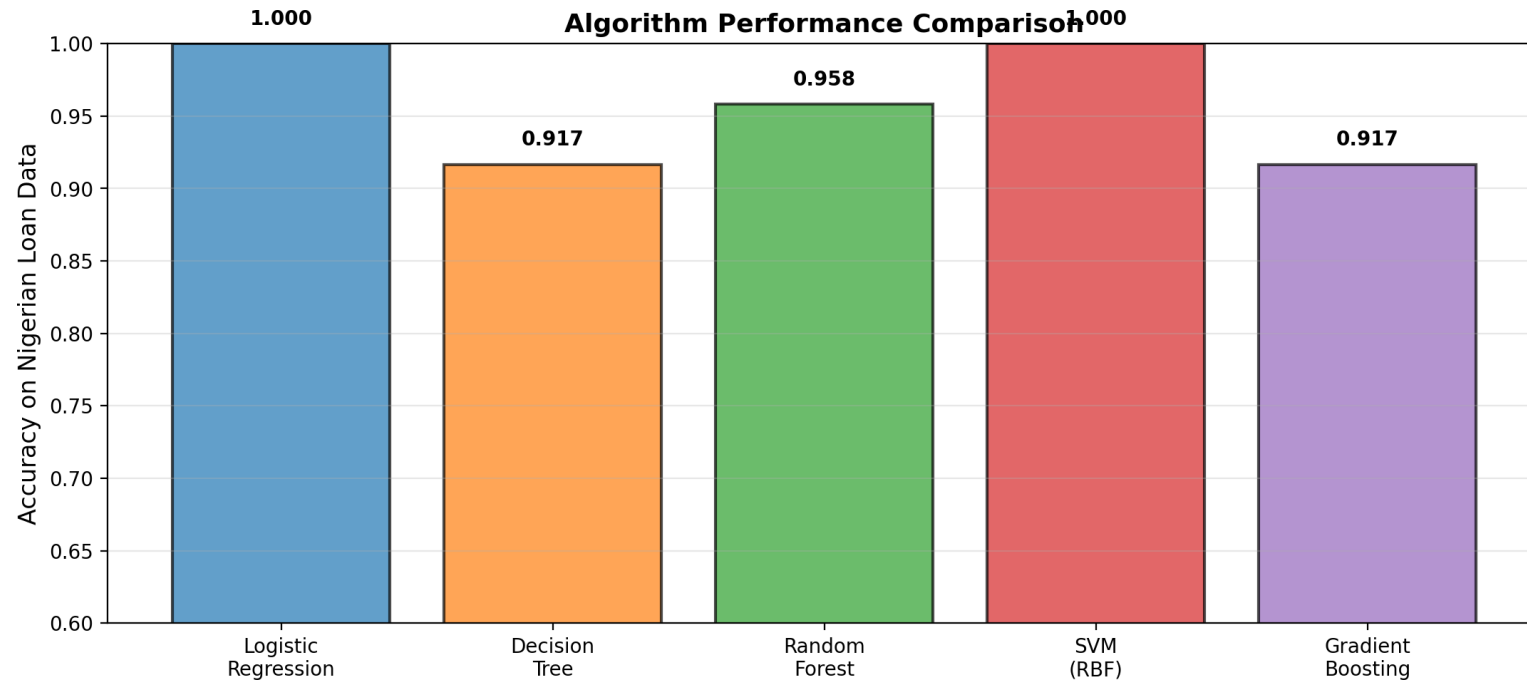
Process:

1. Train first tree on raw data
2. Calculate prediction errors ("residuals")
3. Train next tree to predict those residuals
4. Repeat (typically 50-100 trees)
5. Final prediction = sum of all tree predictions

Pros: Often best performance, captures complex patterns

Cons: Can overfit, slower to train, more hyperparameters

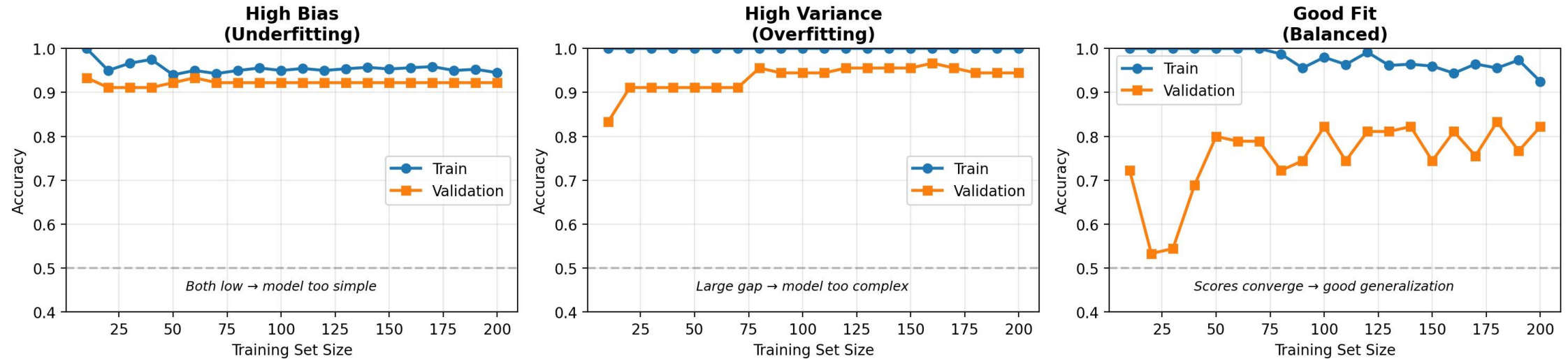
Comparing All Five Algorithms



On Nigerian loan data: All significantly better than random (50%). Gradient Boosting typically highest.

Learning Curves: Diagnosing Performance

Learning Curves: Diagnosing Model Performance

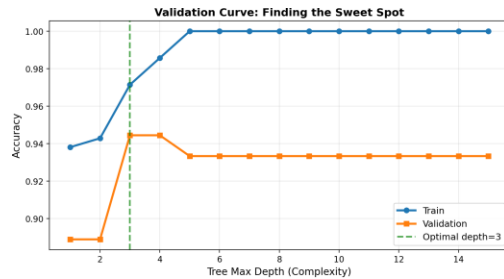


Left panel (High Bias/Underfitting):

- Both train & test accuracy low
- Adding more data won't help → model too simple

Middle panel (High Variance/Overfitting):

Validation Curves: Finding the Sweet Spot

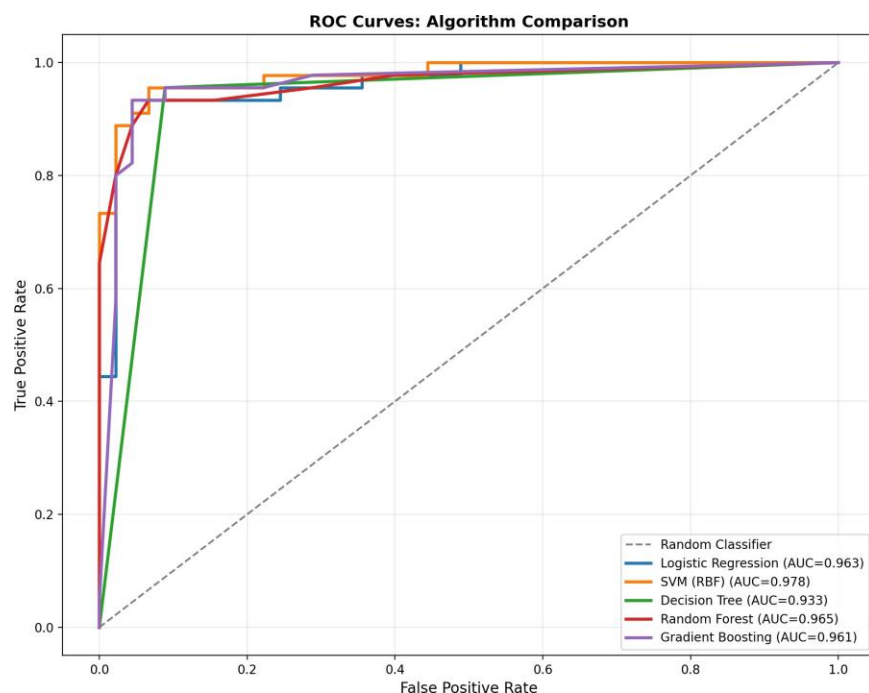


Sweet spot:
validation peaks

How to use: For each hyperparameter value:

1. Train on training set
2. Evaluate on validation set
3. Plot both curves

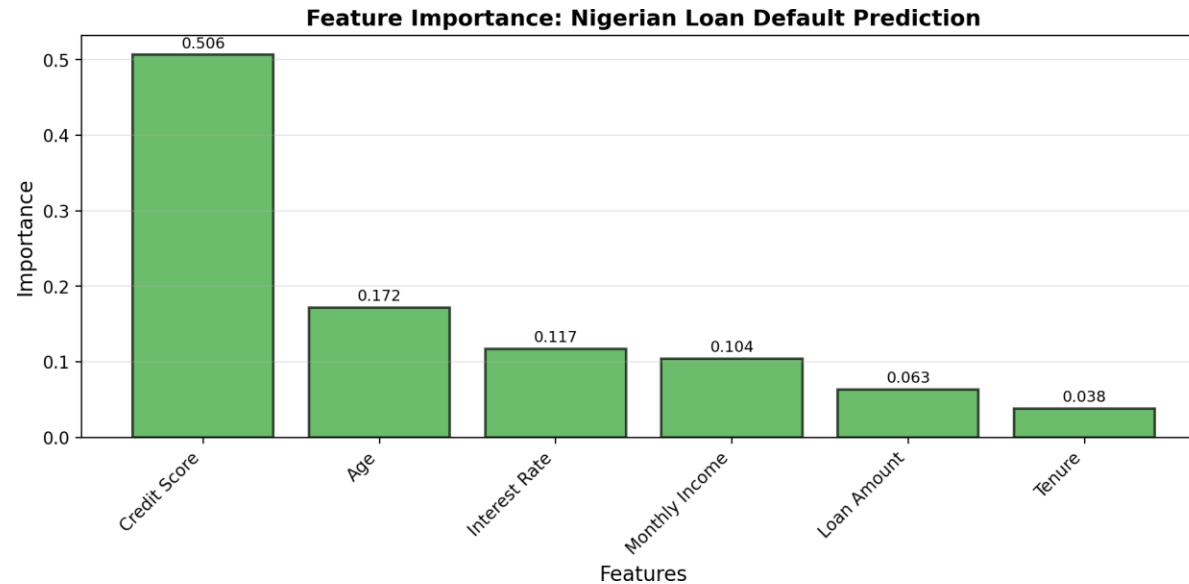
ROC Curves: Measuring Classification Quality



What it shows:

- True Positive Rate (correctly identifying defaults) vs False Positive Rate (incorrectly rejecting good loans)
- Higher curve = better algorithm

Feature Importance: What Did the Model Learn?



Nigerian loan insights:

- Credit score & prior defaults → strongest predictors
- Monthly income & tenure → moderate predictors
- Age → weaker predictor

Brainstorm with Students: Cost-Sensitive Learning

Recall: False Negative (approve bad loan) = **₦500,000 loss**

False Positive (reject good loan) = **₦5,000 loss**

Question for class: Should we treat these errors equally in our model?

Options:

1. Adjust classification threshold (e.g., approve only if $P(\text{default}) < 0.2$ instead of 0.5)
2. Add class weights in training
3. Custom loss function penalizing false negatives more

→ *This is cost-sensitive learning, beyond the scope of Week 4, but essential in practice!*

Model Selection Workflow

- 1. Baseline:** Start with Logistic Regression (fast, interpretable)
- 2. Explore:** Try tree-based (Decision Tree, Random Forest) and ensemble (Gradient Boosting)
- 3. Validate:** Use learning curves to diagnose bias-variance
- 4. Tune:** Validation curves find optimal hyperparameters
- 5. Compare:** ROC curves + AUC for final ranking
- 6. Interpret:** Feature importance explains model decisions

Quick Decision Tree

"Which algorithm should I use?"

Are features mostly numeric & well-structured?

YES → Try Logistic Regression first (baseline)

Does baseline underfitting?

YES → Try non-linear: Decision Tree, Random Forest, SVM

Does model overfitting (learning curve gap)?

YES → Regularize: shallow trees, ensemble, or add data

Need best performance + interpretability tradeoff?

→ Random Forest (good of both worlds)

Need absolute best performance (accept complexity)?

→ Gradient Boosting

High-dimensional data with few samples?

→ SVM with kernel trick

Key Takeaways

1. **Classification $\neq$ Regression:** Use appropriate metrics (confusion matrix, ROC, AUC), not just accuracy
2. **Many algorithms exist:** Each has strengths; choice depends on data, interpretability needs, performance targets
3. **Learning curves diagnose:** High bias $\rightarrow$ model too simple; high variance $\rightarrow$ model too complex
4. **Validation curves optimize:** Find hyperparameter values where validation performance peaks
5. **Cost matters:** Real-world errors have asymmetric costs; algorithms should reflect this
6. **Feature importance reveals:** What the model actually learned; enables business interpretation

Next Steps in Practice Notebook

- **Exercise 1:** EDA on nigerian_loan_default.csv
- **Exercise 2:** Train Logistic Regression baseline
- **Exercise 3:** Decision Tree exploration + learning curves
- **Exercise 4:** Random Forest with feature importance
- **Exercise 5:** SVM training & validation
- **Exercise 6:** Gradient Boosting comparison
- **Exercise 7:** ROC curves & threshold tuning
- **Exercise 8:** Interpreting & presenting results

Questions?

See you in the notebooks!

